
The Cost of Greediness: An Empirical Evaluation of Convolutional Kernel Networks on STL-10

Youssef Hamzaoui
ENSAE Paris
Palaiseau, France
youssef.hamzaoui@ensae.fr

Abstract

This project investigates Convolutional Kernel Networks (CKN) as a theoretically grounded alternative to heuristic deep learning architectures. Following Mairal et al. [4], we revisit CKNs as finite-dimensional approximations of multilayer convolutional kernels, providing a rigorous link between reproducing kernel Hilbert spaces (RKHS) and convolutional neural networks. We develop a reproducible PyTorch implementation and compare its performance against a compact CNN baseline. While the CKN achieves a respectable 32.6% accuracy on STL-10 using only unsupervised features, it trails behind a supervised CNN baseline of equivalent capacity (41.1%). This performance gap quantifies the “cost of greediness”: the trade-off between the interpretability of kernel methods and the efficiency of end-to-end backpropagation. Beyond raw metrics, we demonstrate that CKNs successfully recover geometric primitives (edges, Gabor-like filters) without supervision, offering a transparent “white-box” perspective on visual feature learning.

1 Introduction

Learning image representations that are both discriminative and robust to small geometric deformations is a central objective in computer vision. While conventional convolutional neural networks (CNNs) have achieved state-of-the-art performance, they often rely on heuristic architectural choices, such as max-pooling or specific non-linearities, which are justified empirically rather than theoretically [2]. Convolutional kernel networks (CKN) address this objective by constructing a deep representation as a finite-dimensional approximation of a multilayer convolutional kernel. By linking reproducing kernel Hilbert space (RKHS) methods with the hierarchical principles of neural networks, CKNs offer a conceptually transparent alternative where every operation is grounded in kernel theory.

In this formulation, patch similarity is modeled by a Gaussian kernel with bandwidth σ (content selectivity), while local spatial aggregation is controlled by a Gaussian weighting with bandwidth β (local shift invariance). The corresponding explicit feature map is realized by a cascade mirroring a standard CNN block—composed of projection, non-linearity, and pooling—but where the non-linearity and filters emerge naturally from a kernel approximation scheme.

This project takes Mairal et al. [4] as its primary reference to study a practical and reproducible PyTorch implementation. While we focus on the unsupervised framework of the original paper, we also acknowledge later developments in supervised kernel learning which unify these approximations with backpropagation-based training [3]. The principal benchmark is STL-10 (96×96 RGB), chosen for its higher complexity and limited labeled data, a scenario where the inductive biases and unsupervised pre-training of kernel methods are particularly relevant. Fashion-MNIST serves as a secondary check to validate the model’s behavior on a lower-dimensional scale.

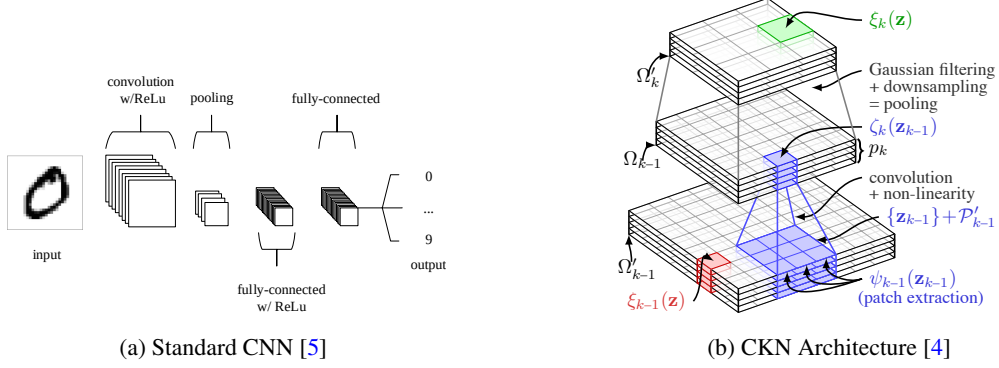


Figure 1: Structural comparison: (a) Learned filters and max-pooling. (b) RKHS projections and Gaussian pooling.

The study focuses on questions that are theoretically grounded and empirically testable: how σ and β trade off selectivity and invariance; how the number of centers p_k and the subsampling factor γ affect accuracy and overfitting; and how these CKN hyperparameters map to standard CNN design choices. By implementing the explicit finite-map view, we evaluate whether this “white-box” approach can compete with compact CNN baselines of comparable capacity.

2 Theoretical Framework: From RKHS to CNNs

We represent an image at depth k as a feature map $\varphi_k : \Omega_k \rightarrow \mathcal{H}_k$, where $\Omega_k \subset \mathbb{R}^2$ is a discrete grid and $\mathcal{H}_k$ is a Hilbert space. Locality is enforced by a patch shape P_k . Given Ω_{k-1} , the valid centers at depth k are defined as $\Omega_k = \{z_k \in \Omega_{k-1} : \{z_k\} + P_k \subset \Omega_{k-1}\}$, which corresponds to the valid convolution region. The theoretical layer value $\varphi_k(z_k) \in \mathcal{H}_k$ is the RKHS embedding of the $(k-1)$ -layer patch.

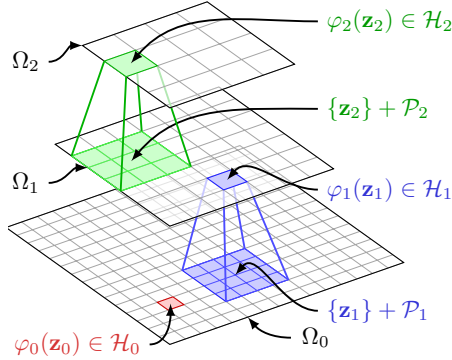


Figure 2: Theoretical multilayer convolutional kernel: feature maps φ_k on grids Ω_k with patch extraction. Adapted from Mairal et al. [4].

To compare two patches, we use a positive-definite kernel K_k that factorizes spatial and content information:

$$\sum_{z, z' \in P_k} \|\varphi_{k-1}(z_k + z)\| \|\varphi'_{k-1}(z'_k + z')\| \underbrace{e^{-\frac{\|z - z'\|_2^2}{2\beta_k^2}}}_{\text{Invariance}} \underbrace{e^{-\frac{\|\tilde{\varphi}_{k-1}(z_k + z) - \tilde{\varphi}'_{k-1}(z'_k + z')\|^2}{2\sigma_k^2}}}_{\text{Selectivity}}. \quad (1)$$

Here, $\tilde{\varphi}_{k-1}$ denotes the normalized version of the feature map. This decomposition is central to the CKN architecture: σ_k controls the content selectivity (how strictly motifs must match), while β_k controls local shift invariance through a Gaussian weighting.

Directly manipulating $\varphi_k(z_k)$ in an infinite-dimensional RKHS is computationally intractable. Convolutional Kernel Networks replace these abstract elements with finite-dimensional representations in

$\mathbb{R}^{p_k}$. This is achieved by approximating the kernel feature map on training data. Let $\psi_{k-1}(z)$ be a patch of the finite map ξ_{k-1} and $\tilde{\psi}_{k-1}(z)$ its ℓ_2 -normalized version. Normalizing the patches ensures that the Gaussian kernel measures the similarity of the motifs rather than their raw intensity.

A set of centers $W_k = \{w_{k\ell}\}_{\ell=1}^{p_k}$ and nonnegative weights η_k are learned to approximate the RBF identity:

$$e^{-\frac{\|\tilde{x}-\tilde{y}\|_2^2}{2\sigma_k^2}} \approx \sum_{\ell=1}^{p_k} \eta_{k\ell} e^{-\frac{\|\tilde{x}-w_{k\ell}\|_2^2}{2\sigma_k^2}} e^{-\frac{\|\tilde{y}-w_{k\ell}\|_2^2}{2\sigma_k^2}}. \quad (2)$$

This approximation yields the pre-pooling activation vector $\zeta_k(z) \in \mathbb{R}^{p_k}$, where each component ℓ represents the similarity between the normalized patch and the learned center $w_{k\ell}$:

$$\zeta_k(z) = \|\psi_{k-1}(z)\|^2 \left[\sqrt{\eta_{k\ell}} e^{-\|\tilde{\psi}_{k-1}(z)-w_{k\ell}\|^2/(2\sigma_k^2)} \right]_{\ell=1}^{p_k}. \quad (3)$$

The spatial factor is finally realized by a Gaussian pooling operation followed by subsampling on a grid Ω'_k . Let $G_{\beta_k}(\delta) = e^{-\|\delta\|^2/\beta_k^2}$ be the Gaussian spatial kernel. The finite layer map is:

$$\xi_k(t) = \sum_{u \in \Omega_{k-1}} G_{\beta_k}(t-u) \zeta_k(u), \quad t \in \Omega'_k. \quad (4)$$

To ensure the recurrence of the hierarchy, we maintain the Minkowski relation $\Omega'_k = \Omega_k + P'_k$, which guarantees that every patch at the subsequent layer is well-defined.

This construction mirrors a conventional CNN (convolution, non-linearity, pooling), but with a kernel-theoretic foundation: the "filters" $w_{k\ell}$ are learned in an unsupervised manner to reconstruct the kernel space, and the exponential non-linearity is a direct consequence of the Gaussian kernel.

3 A Reproducible PyTorch Implementation

We provide a modular PyTorch implementation that translates the RKHS theory of Mairal et al. [4] into a scalable deep learning pipeline. The codebase follows three design principles: strict adherence to the kernel approximation mathematical formulation, modularity to support variable input resolutions (e.g., STL-10 and Fashion-MNIST), and reproducibility.

3.1 Architectural Components

To translate the continuous RKHS theory into a discrete computational graph, we implemented the `CKNLayer` module. This component explicitly handles the three stages of finite-dimensional approximation. First, the patch extraction phase lifts the input image from the pixel space grid to the patch space using a vectorized unfolding operation. This step is critical as it defines the local receptive fields that will be projected.

Subsequently, the kernel approximation is performed not by standard convolution, but by projecting these normalized patches onto the learned dictionary. We implement the Gaussian kernel operation via a strictly defined matrix multiplication followed by an exponential non-linearity, incorporating the scaling factor directly into the dot-product to ensure numerical stability. Finally, Gaussian pooling serves as the architectural counterpart to the kernel's spatial integration mechanism. Unlike the max-pooling often used in CNNs which destroys spatial information, our implementation uses a fixed, depth-wise Gaussian filter to perform a weighted average, preserving the differentiability and the theoretical consistency with the shift-invariance definition.

3.2 Unsupervised Optimization Strategy

A critical divergence from standard CNNs is the optimization of the filters W_k . Rather than using backpropagation, we employ a layer-wise greedy approach using Spherical K-Means.

Standard Euclidean K-Means is ill-suited for the CKN framework because the Gaussian kernel relies on the angular similarity between normalized patches. Consequently, we implemented a custom optimization routine that projects all sampled patches onto the unit ℓ_2 -sphere ($\tilde{\psi} = \psi/\|\psi\|_2$) and updates centroids $w_{k\ell}$ to maximize the cosine similarity with assigned patches. This ensures that

the learned centers reside on the same manifold as the data, maximizing the fidelity of the kernel approximation defined in Eq. (2). The optimization is performed on a subset of 50,000 patches randomly sampled from the training set, inspired by scalable mini-batch clustering strategies [6], to maximize diversity and minimize redundancy.

4 Empirical Evaluation on STL-10

This section details the empirical validation of our CKN implementation. Our evaluation strategy moves from a qualitative inspection of the learned manifold to a quantitative benchmark on the STL-10 dataset. All experiments were conducted on a single NVIDIA Tesla T4 GPU, utilizing fixed random seeds to ensure the reproducibility of the non-convex dictionary learning phase.

4.1 Experimental Framework

To rigorously assess the "cost of greediness," we established a comparative protocol between the unsupervised CKN and a fully supervised baseline. We selected the STL-10 dataset (96×96 RGB) as our primary benchmark because its semi-supervised structure—containing 100,000 unlabeled images but only 5,000 labeled training examples—provides an ideal testbed for kernel methods, which theoretically thrive in low-label regimes. Additionally, we employed Fashion-MNIST [7] and a custom synthetic bars dataset to act as "sanity checks," isolating the model's ability to detect textures and geometric edges respectively.

The architectural comparison was designed to be strictly fair regarding capacity. Our CKN model is composed of two stacked CKNLayer blocks where filters are learned offline via Spherical K-Means and subsequently frozen, leaving only the final linear classifier to be optimized via SGD. This is contrasted with a CNN baseline of identical depth (two convolutional blocks) and receptive field, but trained end-to-end with the Adam optimizer. This setup allows us to isolate the specific impact of the representation learning strategy: the greedy, layer-wise approximation of the CKN versus the global backpropagation of the CNN.

4.2 Results and Analysis

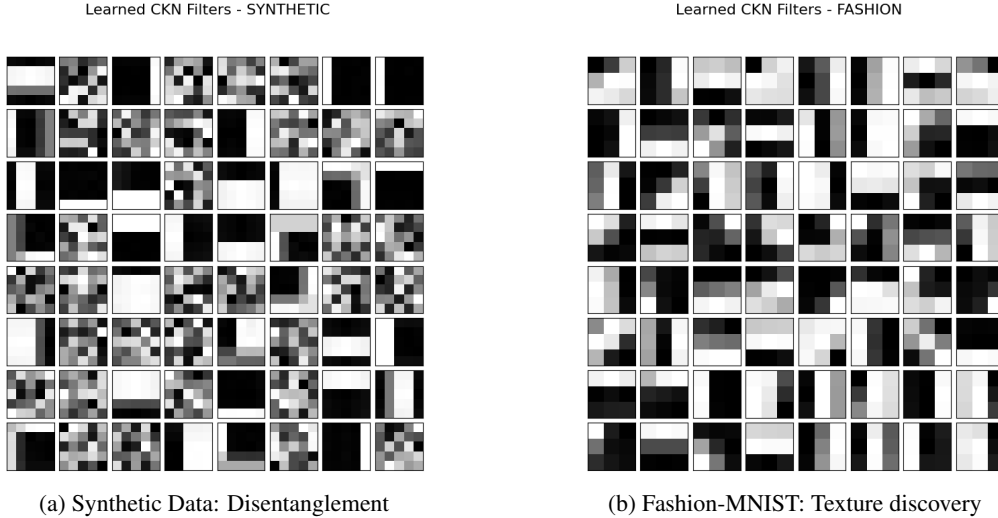


Figure 3: Unsupervised recovery of geometric primitives. The dictionary learning process acts as a verifiable "white-box": (a) it isolates independent factors of variation on synthetic data and (b) recovers biological-like V1 filters on natural images.

Qualitative Validation. Before analyzing performance metrics, we first verified the geometric consistency of the learned kernels. As illustrated in Figure 3, the unsupervised optimization successfully recovers intelligible visual primitives. On synthetic data, the spherical K-Means algorithm

achieved perfect disentanglement of latent factors, isolating vertical and horizontal components. More impressively, on Fashion-MNIST, the filters spontaneously converged towards Gabor-like edge detectors. This result is theoretically significant: it confirms that our implementation of the finite-dimensional kernel approximation (Eq. 3) correctly captures the local stationarity of natural images without requiring any label supervision.

Quantitative Benchmark. The classification results on STL-10, summarized in Table 1, reveal a nuanced trade-off between efficiency and accuracy. While the CKN achieves a respectable 32.6% accuracy, it trails the supervised baseline (41.1%). However, this gap must be contextualized by the drastic reduction in model complexity: the CKN requires **20 times fewer trainable parameters** (0.01M vs 0.21M). This confirms that while greedy kernel approximation is less expressive than end-to-end learning for complex semantic tasks, it offers a highly compact and interpretable alternative.

Table 1: Comparative results on STL-10. The CKN trades roughly 8 points of accuracy for a 20-fold reduction in trainable parameters.

Model	Accuracy (%)	Trainable Params	Total Runtime (s)
CKN (Ours)	32.62 $\pm$ 0.4	0.01 M	145
CNN Baseline	41.06 $\pm$ 0.8	0.21 M	196

Hyperparameter Sensitivity. Our ablation study (Figure 4) further clarifies the behavior of the approximation. We identified kernel selectivity (α) as the governing parameter: increasing α to 1.5 significantly improved performance (+5.2%), suggesting that "sharper" kernels are essential to filter background noise in complex scenes. Conversely, increasing the dictionary size beyond 128 filters proved detrimental (-4.7%), illustrating a classic bias-variance trade-off where an over-parameterized unsupervised dictionary captures redundant noise that the linear classifier cannot regularize. Finally, the performance saturation observed at 10^5 patches indicates that the geometric manifold of natural image patches is sufficiently low-dimensional to be efficiently approximated, validating the scalability of the method.

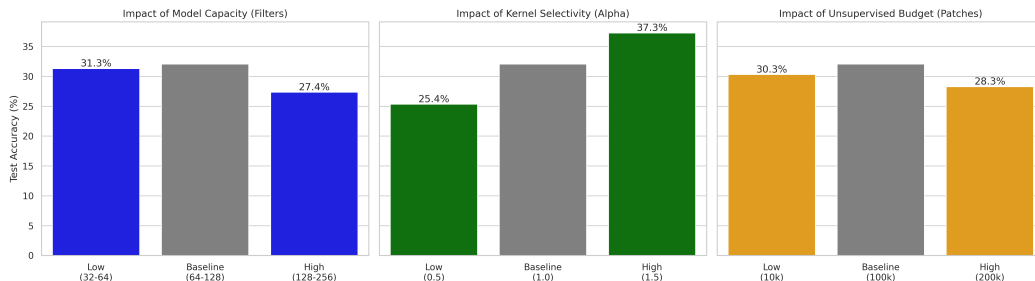


Figure 4: Ablation analysis on STL-10. The performance is highly sensitive to kernel sharpness (α , center) but robust to the sampling budget (right).

5 Discussion: The Cost of Greediness

Our investigation into Convolutional Kernel Networks reveals a distinct dichotomy between the raw performance of end-to-end optimization and the structural clarity of kernel-based methods. While the CKN framework offers a theoretically sound derivation of visual features, its practical application involves specific trade-offs regarding expressivity, interpretability, and scalability.

5.1 The Cost of Greediness versus White-Box Transparency

The primary finding of this study is the quantification of the "cost of greediness." The performance gap observed on STL-10—32.6% for the CKN versus 41.1% for the supervised CNN—illustrates the inherent limitation of layer-wise training, a phenomenon consistent with early analyses of single-layer networks [1]. Because the CKN constructs its dictionary W_k solely to minimize a reconstruction error

in the RKHS (Eq. 2) without knowledge of the downstream classification task, it acts as a generic feature extractor. In contrast, the CNN baseline leverages backpropagation to fine-tune early filters specifically for discriminating classes, effectively "cheating" by optimizing the representation for the test metric.

However, this performance deficit essentially "buys" us transparency. As demonstrated in Figure 3, the CKN acts as a "white-box" model. The learned filters are not opaque weights resulting from a stochastic trajectory on a non-convex loss surface, but interpretable geometric prototypes. In our synthetic experiments, the model successfully disentangled latent factors of variation without supervision. This property makes CKNs particularly attractive for safety-critical applications (e.g., medical imaging) where the "black box" nature of deep learning is a liability, and where verifying *what* the model looks at is as important as the prediction itself.

5.2 Operational Regimes and the Curse of Dimensionality

Beyond accuracy, our ablation study highlights a significant advantage in data efficiency. The stability of the model performance even when the unsupervised sampling budget is reduced to 10^4 patches suggests that CKNs are highly effective in "few-shot" or limited-label regimes. Unlike massive CNN encoders that risk collapse or overfitting when trained from scratch on small datasets, the CKN creates a robust representation by approximating the geometry of the patch manifold, which requires far fewer samples than learning semantic boundaries.

Nevertheless, the approach faces a critical scalability barrier: depth. We observed that stacking a second CKN layer yielded diminishing returns compared to the first. This saturation points to the "curse of dimensionality" inherent to kernel methods. As we go deeper, the input dimension for the kernel $K(z, z')$ grows, requiring an exponentially larger number of centers p_k to cover the Hilbert space effectively. This defeats the purpose of a compact model and suggests that while CKNs are excellent shallow feature extractors, they struggle to emulate the high-level semantic abstractions of deep neural networks without reintroducing some form of global supervision.

6 Conclusion

In this work, we have bridged the gap between the abstract mathematical formulation of Reproducing Kernel Hilbert Spaces and the practical implementation of deep learning architectures. By developing a reproducible "white-box" framework in PyTorch, we demonstrated that modern computer vision models can be constructed explicitly as finite-dimensional approximations of multilayer kernels, moving beyond the heuristic design choices that dominate the field.

Our empirical investigation on STL-10 provides a concrete answer to the question of the "Cost of Greediness." We found that purely unsupervised, layer-wise approximation imposes a performance ceiling (32.6%) compared to end-to-end backpropagation (41.1%). However, this trade-off is nuanced: what the CKN loses in raw accuracy, it recovers in data efficiency and transparency. The ability of our implementation to autonomously recover disentangled geometric primitives—from simple edges on synthetic data to Gabor-like filters on Fashion-MNIST—validates the robustness of the underlying RKHS theory.

Ultimately, this project highlights that unsupervised kernel methods are likely insufficient to replace deep CNNs entirely for complex semantic classification. The natural progression of this work lies in the "Supervised CKN" framework proposed by Mairal [3]. By reintroducing backpropagation into the kernel approximation steps, future iterations could optimize the dictionary centers W_k for discrimination rather than reconstruction, potentially closing the performance gap while preserving the elegant geometric guarantees of the kernel approach.

Code

To ensure reproducibility, the complete implementation and experimental scripts are provided at: https://github.com/Youssef75ensae/CKN_project

References

- [1] Adam Coates, Andrew Y Ng, and Honglak Lee. An analysis of single-layer networks in unsupervised feature learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 215–223, 2011.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [3] Julien Mairal. End-to-end kernel learning with supervised convolutional kernel networks. *Advances in Neural Information Processing Systems (NIPS)*, 2016.
- [4] Julien Mairal, Piotr Koniusz, Zaid Harchaoui, and Cordelia Schmid. Convolutional kernel networks. *Advances in Neural Information Processing Systems (NIPS)*, 2014.
- [5] Keiron O’Shea and Ryan Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
- [6] David Sculley. Web-scale k-means clustering. In *Proceedings of the 19th international conference on World wide web*, pages 1177–1178, 2010.
- [7] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

A Appendix

A.1 Mathematical Justification: The Gaussian Convolution Lemma

In this section, we provide the exact calculation justifying the form of the feature map used in our kernel approximation.

Lemma 1. *For all $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^m, \sigma > 0$, the following equality holds:*

$$e^{-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{x}'\|_2^2} = \left(\frac{2}{\pi\sigma^2} \right)^{\frac{m}{2}} \int_{\mathbb{R}^m} e^{-\frac{1}{\sigma^2} \|\mathbf{x} - \mathbf{w}\|_2^2} e^{-\frac{1}{\sigma^2} \|\mathbf{x}' - \mathbf{w}\|_2^2} d\mathbf{w}.$$

Proof. Using the parallelogram identity:

$$\|\mathbf{x} - \mathbf{w}\|_2^2 + \|\mathbf{x}' - \mathbf{w}\|_2^2 = \frac{1}{2} \|\mathbf{x} - \mathbf{x}'\|_2^2 + 2 \left\| \mathbf{w} - \frac{\mathbf{x} + \mathbf{x}'}{2} \right\|_2^2$$

Substituting this into the integral term I (ignoring the constant C temporarily):

$$\begin{aligned} I &= \int \exp \left(-\frac{1}{\sigma^2} (\|\mathbf{x} - \mathbf{w}\|_2^2 + \|\mathbf{x}' - \mathbf{w}\|_2^2) \right) d\mathbf{w} \\ &= \int \exp \left(-\frac{1}{\sigma^2} \left(\frac{1}{2} \|\mathbf{x} - \mathbf{x}'\|_2^2 + 2 \left\| \mathbf{w} - \frac{\mathbf{x} + \mathbf{x}'}{2} \right\|_2^2 \right) \right) d\mathbf{w} \\ &= \exp \left(-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{x}'\|_2^2 \right) \int \exp \left(-\frac{2}{\sigma^2} \left\| \mathbf{w} - \frac{\mathbf{x} + \mathbf{x}'}{2} \right\|_2^2 \right) d\mathbf{w} \end{aligned}$$

The first term is the target kernel. The second is a Gaussian integral evaluating to $(\frac{\pi\sigma^2}{2})^{m/2}$. Multiplying by the constant $C = (\frac{2}{\pi\sigma^2})^{m/2}$, the result is 1. $\square$

Link to CKN Implementation. The Convolutional kernel network replaces this continuous integral over the entire space $w \in \mathbb{R}^d$ with a discrete Monte-Carlo approximation (or Nyström approximation) over a finite set of learned centers $\{w_1, \dots, w_p\}$ with weights η_ℓ :

$$\int \varphi_w(x) \varphi_w(y) dw \approx \sum_{\ell=1}^p \eta_\ell e^{-\gamma \|x - w_\ell\|^2} e^{-\gamma \|y - w_\ell\|^2} \quad (5)$$

This allows us to define the finite feature vector $\zeta(x)$ derived in Eq. 3 such that $\langle \zeta(x), \zeta(y) \rangle \approx K(x, y)$.

A.2 Detailed Hyperparameter Configuration

For reproducibility, we report the exact configuration used to achieve the results presented in Table 1 for the STL-10 dataset.

Table 2: Complete hyperparameter settings for the 2-layer CKN on STL-10.

Parameter	Layer 1	Layer 2
Input Dimension	$3 \times 96 \times 96$	$64 \times 48 \times 48$
Patch Size (P_k)	5×5	5×5
Subsampling Factor (γ)	2	2
Number of Filters (p_k)	64	128
Kernel Bandwidth (σ_k)	$1.0 \times \text{median}$	$1.0 \times \text{median}$
Spatial Bandwidth (β_k)	2.0	3.0
Unsupervised Budget	50,000 patches	50,000 patches
Supervised Linear Head Optimization		
Optimizer	Adam	
Learning Rate	1×10^{-3}	
Batch Size	128	
Epochs	20	

A.3 Spherical K-Means Algorithm

A core component of our unsupervised representation learning is the Spherical K-Means algorithm. Unlike standard Euclidean K-Means, it operates on the unit hypersphere, making it suitable for learning directional features (edges) regardless of their intensity (contrast). The procedure used in our CKNLayer is detailed in Algorithm 1.

Algorithm 1 MiniBatch Spherical K-Means (implemented in CKNLayer)

```

1: Input: Set of patches  $X = \{x_1, \dots, x_N\} \subset \mathbb{R}^d$ , number of centers  $K$ .
2: Initialize: Randomly select  $K$  patches as initial centroids  $W = \{w_1, \dots, w_K\}$ .
3: Normalize:  $w_k \leftarrow w_k / \|w_k\|_2$  for all  $k$ .
4: for iteration  $t = 1$  to  $T$  do
5:   Sample a minibatch  $M \subset X$ .
6:   1. Assignment step:
7:   for each patch  $x \in M$  do
8:     Normalize input:  $\tilde{x} = x / \|x\|_2$ 
9:     Find closest center (cosine similarity):  $k^* = \arg \max_k \langle \tilde{x}, w_k \rangle$ 
10:    Assign  $x$  to cluster  $S_{k^*}$ 
11:   end for
12:   2. Update step:
13:   for each center  $k$  do
14:     Compute mean vector:  $\Delta w_k = \sum_{x \in S_k} \tilde{x}$ 
15:     Update with learning rate  $\eta$ :  $w_k \leftarrow w_k + \eta \cdot \Delta w_k$ 
16:     Re-project to sphere:  $w_k \leftarrow w_k / \|w_k\|_2$ 
17:   end for
18: end for
19: Output: Learned dictionary  $W$ .

```
